

UPython 3 Pro Document

[Two Cats Code](#) | [Online Documentation](#) | [Discord](#) | [Unity Asset-Store](#)

Intro

Tired of using Python scripts in Unity? UPython 3 Pro may be able to achieve what you need.

UPython 3 Pro is a tool out of the box, as long as you have configured the Python environment.

❗ Not compatible with UPython 2 (Pro).

UPython 3 Pro redesigns the underlying logic of Unity component and Python server. A highly integrated and optimized design makes it easier for developers to use or extend remote Python calls.

Tired of using Python scripts in Unity? UPython 3 Pro may be able to achieve what you need.
UPython 3 Pro is a tool out of the box, as long as you have configured the Python environment.
❗ Not compatible with UPython 2 (Pro).
UPython 3 Pro redesigns the underlying logic of Unity component and Python server. A highly integrated and optimized design makes it easier for developers to use or extend remote Python calls.
Just start one Python server, and it will provide all the functionality.
Unity and Python Server could run on the same PC or run on different PCs that the PCs should stay within the same local network (e.g., connect to the same wifi)
[Release](#)
[Tutorials & Tips](#)
Custom Order-Sensitive Call Function
Please cite this tool if you used it in a research paper.

Features

- 👍 All [UPython 2 Pro](#) users can free upgrade to UPython 3 Pro.
- 👍 All UPython 3 Pro users will also have [UPython 2 Pro](#) for FREE!
- Included all the py server scripts.
- Source code included.
- Easy to use; Zero learning cost.
- Callback event supports.
- Async call supports.
- 🟢 3 basic calls:

🟢 (P) Call preset python script on the Python Server and send the result to

- 🔥 All [UPython 2 Pro](#) users can free upgrade to UPython 3 Pro.
- 🔥 All UPython 3 Pro users will also have [UPython 2 Pro](#) for FREE!
- Included all the py server scripts.
- Source code included.
- Easy to use; Zero learning cost.
- Callback event supports.
- Async call supports.
- 🟢 3 basic calls:
 - 🟢 (P) Call preset python script on the Python Server and send the result to Unity.
 - 🟢 (LTP) Big file transfer (<4MBs) from Unity to Python Server.
 - 🟢 (PTU) Big file transfer (<4MBs) from Python Server to Unity.
 - 🟢 (C) Composite call:
 - 🟢 (LTP) (P) Write python script in Unity C#, run it on the Python Server directly, and get the result from Python Server.
- Run in both Play and Editor modes.
- Custom connection life cycle:
 - 🟢 The default life cycle of each connection is one Python Call. It automatically manages all connections without manual management.

Installation

- Install via package manager in Unity editor.
- You can move the [Python] folder to anywhere outside the [Asset] folder.

Validate python installation:

- Start the Python server: (use CMD on Win or use Terminal on Mac, Linux also support)
- You will see:

Then the python server is launched.

To Exit the python server, you can use `Ctrl + C` to stop.

• Install via package manager in Unity editor.
• You can move the [Python] folder to anywhere outside the [Asset] folder.
Validate python installation:
1. Start the Python server: (use CMD on Win or use Terminal on Mac, Linux also support)

```
python_server.py
```


2. You will see:

```
Server Started!  
Listening...
```


Then the python server is launched.
To Exit the python server, you can use `Ctrl + C`

```
exit()
```


When you exit from the server, you may have an error; you can ignore it. It was caused by Python 3.x, and it would not be a trouble for us.
Validate Unity connection:
1. Open the scene: [Assets/TwoCatsCode/](#)

Please cite this tool if you used it in a research paper.

```
@misc{
  huang_UP3P_2022,
  tool=(UPython 3 Pro),
  author=(Huang, Haikun), year=(2022)
}
```



Release

1.0

- Initial.



Tutorials & Tips

Launch python server:

1. Start the Python server: (use



Custom Order-Sensitive Call Function

An order-Sensitive Call is an advanced operation. It allows us to

↑ [UPython 3 Pro Document](#)

Intro

Tired of using Python scripts in Unity? UPython 3 Pro may be able to achieve what you need.

UPython 3 Pro is a tool out of the box, as long as you have configured the Python environment.

❗ Not compatible with UPython 2 (Pro).

UPython 3 Pro redesigns the underlying logic of Unity component and Python server. A highly integrated and optimized design makes it easier for developers to use or extend remote Python calls.

Just start one Python server, and it will provide all the functionality.

Unity and Python Server could run on the same PC or run on different PCs that the PCs should stay within the same local network (e.g., connect to the same wifi)

[Release](#)

[Tutorials & Tips](#)

[Custom Order-Sensitive Call Function](#)

[Please cite this tool if you used it in a research paper.](#)

↑ [UPython 3 Pro Document](#)

Features

- 👍 All [UPython 2 Pro](#) users can free upgrade to UPython 3 Pro.
- 👍 All UPython 3 Pro users will also have [UPython 2 Pro](#) for **FREE!**
- Included all the py server scripts.
- Source code included.
- Easy to use; Zero learning cost.
- Callback event supports.
- Async call supports.

-  3 basic calls:
 - (P) Call preset python script on the Python Server and send the result to Unity.
 - (UTP) Big file transfer (<4MBs) from Unity to Python Server.
 - (PTU) Big file transfer (<4MBs) from Python Server to Unity.
-  1 composite call:
 - (UTP, P) Write python script in Unity C#, run it on the Python Server directly, and get the result from Python Server.
- Run in both Play and Editor modes.
- Custom connection life cycle:
 - The default life cycle of each connection is one Python Call. It automatically manages all connections without manual management.
 - Custom connection lifecycle can support multiple Python Calls until manually closed. Suitable for those order-sensitive python calls.
- Extensible python script calling order.
- No 3rd party tool is required!

↑ UPython 3 Pro Document

Installation

- Install via package manager in Unity editor.
- You can move the [Python] folder to anywhere outside the [Asset] folder.

Validate python installation:

1. Start the Python server: (use CMD on Win or use Terminal on Mac, Linux also support)

```
python server.py
```

2. You will see:

```
Server Started!
Listening...
```

Then the python server is launched.

To Exit the python server, you can use `Ctrl + c` or:

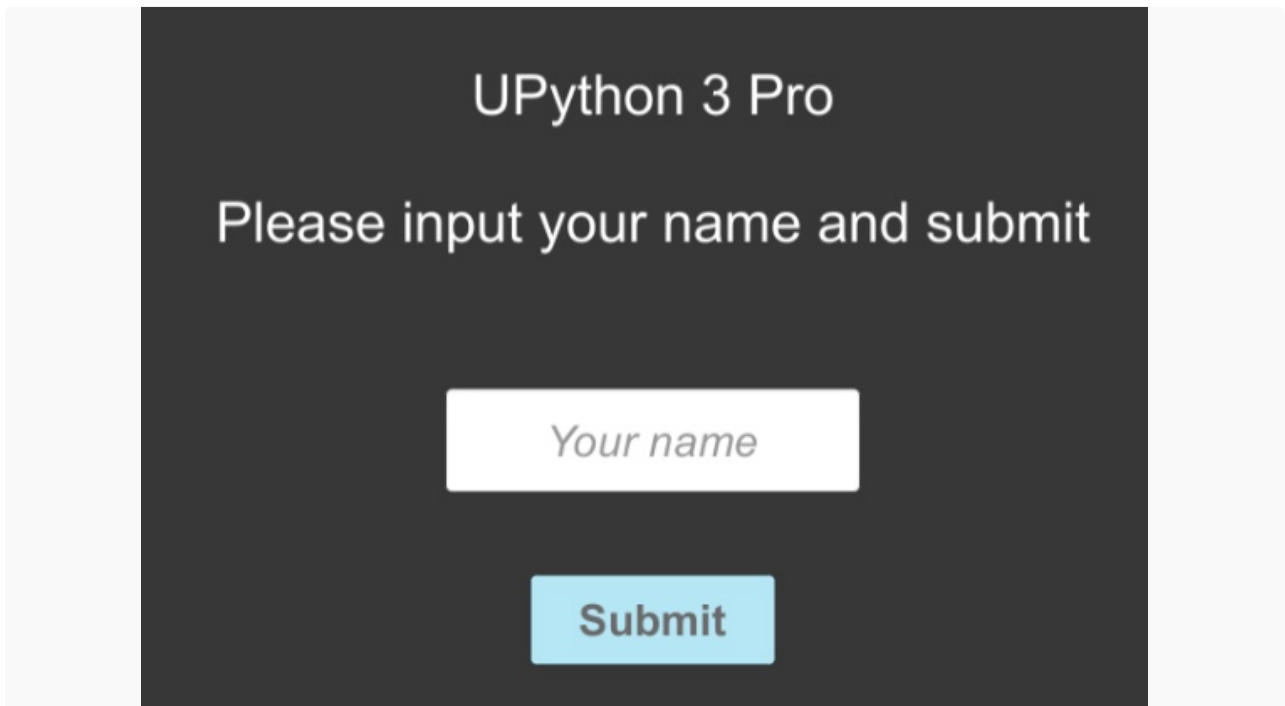
```
exit()
```

When you exit from the server, you may have an error; you can ignore it. It was caused by Python 3. x, and it would not be a trouble for us.

Validate Unity connection:

1. Open the scene `Assets/TwoCatsCode/UPython 3 Pro/Demo/1.Hello/1.Hello.unity`

2. Enter [Play Mode](#).
3. Input your name and submit it.
4. See the result.



↑ [UPython 3 Pro Document](#)

Please cite this tool if you used it in a research paper.

```
@misc{
  huang_UP3P_2022,
  tool={UPython 3 Pro},
  author={Huang, Haikun}, year={2022}
}
```

↑ [UPython 3 Pro Document](#)

Release

1.0

- Initial.

↑ [UPython 3 Pro Document](#)

Tutorials & Tips

Launch python server:

1. Start the Python server: (use CMD on Win or use Terminal on Mac, Linux also support)

```
python server.py
```

2. You will see:

```
Server Started!  
Listening...
```

Then the python server is launched.

Please see the demos:

- Assets/TwoCatsCode/UPython 3 Pro/Demo/***

Create UPython Setting SO

- Right-click in the project window: `Two Cats Code/UPython 3 Pro/UPython Setting`
- **Host**: the python server's IP
- **Port**: the python server's port
- **Data Buffer**: the maximum bytes the data send and recv. It is system dependance. By default, it is 4MBs.
- **End_signal**: an identify of the end signal. Should match server.py settings.
- **Exit_cmd**: an identify of the exit command. Should match server.py settings.
- **Send Interval**: The interval of sending the data from Unity to Python Server. Should increase the interval if you have a data stream problem during big file transfer.
- **Show Log**: Debug use, to show the log to the console.

Script	# UPythonSetting
Host	127.0.0.1
Port	8888
Data Buffer	4194304
End_signal	[]
Exit_cmd	exit()
Send Interval In Milli s	50
Show Log	☒

Key APIs

All functions include callback events and async functionalities.

- The `result` could be null if you do not need the callback.

3 Basic Functions

```
// Call Preset python script
public async Task<string> CallPreset(string command, UnityAction<string> result = null)

// Bytes Python to Unity
public async Task<byte[]> CallPUT(string filePath, UnityAction<byte[]> result = null)

// Bytes Unity to Python
public async Task<string> CallUTP(string filePath, byte[] sendingBytes,
UnityAction<string> result = null)
```

Composite Function

```
// Run raw python script
public async Task<string> CallPythonScript(string saveToFilePath, string script,
UnityAction<string> result = null)
```

Tips

It depends how you call this function. The `result` event could be null. For more details, please see the demos.

Custom Order-Sensitive Call Function

An order-Sensitive Call is an advanced operation. It allows us to achieve flexible and complex results by running preset python scripts in a sequential combination. For example, run script A, get the result as the input of script B, and then the result of script B as the input of script C.

A practical example is object recognition. First, capture the screen in Unity, then send the screenshot as an image file to the Python server via CallUTP. After the Python server accepts the image file and saves it locally, call the Python script via CallPreset to recognize the image and then return the result to Unity.

UPython 3 Pro supports custom calling orders and is extremely easy to implement.

First

All `UPython` classes should derive from `UPythonBase` as `UPythonBase` provides all the low-level and core functions. In addition, all custom calls can achieve complex functionality by combining those low-level APIs.

For more details, please see `UPython.cs` and `UPythonBase.cs`

Learn 3 basic calls.

```
// Call Preset python script
protected async Task<string> _Preset(string command, UnityAction<string> result = null,
UPythonConnection externalConnection = null)

// Bytes Python to Unity
protected async Task<UPythonConnection.ReceiveData> _PTU(string filePath,
UnityAction<UPythonConnection.ReceiveData> resultBytes = null, UPythonConnection
externalConnection = null)

// Bytes Unity to Python
protected async Task<string> _UTP(string filePath, byte[] sendingBytes,
UnityAction<string> result = null, UPythonConnection externalConnection = null)
```

Learn 1 composite call.

```
// Run raw python script
protected async Task<string> _PythonScript(string saveToFilePath, string script,
UnityAction<string> result = null, UPythonConnection externalConnection = null)
```

Create connection

```
// Create a persistent connection, need to manually close it. Return the connection that
has been connected to the python server.
public static async Task<UPythonConnection> CreateConnection(UPythonSetting setting,
UnityAction<UPythonConnection> result = null)

// Close a connection
public async Task Close()
```

Example: Order-Sensitive Call Function

```

// Run raw python script
// DOC: This is a good example of learning how to finish a list of tasks using one
socket connection for the calling order-sensitive tasks.
// Create an external connection, connect to the python server, run the python tasks
with the external connection, then close the connection.

protected async Task<string> _PythonScript(string saveToFilePath, string script,
UnityAction<string> result = null, UPythonConnection externalConnection = null)
{
    UPythonConnection c;
    if (externalConnection != null)
        c = externalConnection;
    else
    {
        var tc = UPythonConnection.CreateConnection(setting, null);
        await Task.WhenAll(tc);
        c = tc.Result;
    }

    // Save the script to python side
    var t = _UTP(saveToFilePath, Encoding.UTF8.GetBytes(script), null, c);
    await Task.WhenAll(t);

    // Run the script as a preset
    t = _Preset(saveToFilePath, null, c);
    await Task.WhenAll(t);

    // Close local socket
    if (externalConnection == null)
        await c.Close();

    // Get result
    if(result != null)
        result.Invoke(t.Result);

    return t.Result;
}

```